Competition Topic 1: Icy Grid World

Codex use is required. Use Codex to help implement and review the environment, but treat its output as an untrusted draft. You are responsible for every line you submit, for testing the stochastic behavior, and for explaining how the code implements the MDP below. Record your work in `AI_LOG.md`.

1 Story and task

An explorer starts in the southwest corner of a frozen 20×20 field. The gold is in the northeast corner. Rocks are impassable. On each step the explorer chooses a cardinal direction, but the ice may send the explorer perpendicular to the intended direction. The objective is to reach the gold while taking as few steps as possible.

Implement this environment as a reusable Python package named `icy_gridworld`. The registered environment ID must be `IcyGridWorld-v0`. This handout is the authoritative specification; do not silently replace any constants or conventions with those of another Grid World.

2 The MDP

Following the notation used in the Week 2 lecture, the task is the discounted episodic MDP

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle.$$

Here S_t, A_t, and R_{t+1} denote the state, action, and immediate reward at time-step t. For $s, s' \in \mathcal{S}$ and $a \in \mathcal{A}$, the lecture notation for transition probabilities and expected immediate rewards is

$$P^a_{ss'} = \mathbb{P}[S_{t+1} = s' \mid S_t = s, A_t = a], \qquad R^a_s = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a].$$

All randomness is in $\mathcal{P}$. The map, starting state, and goal are fixed.

2.1 Coordinates, states, start, and goal

Let

$$\mathcal{C} = \{(r, c) : r, c \in \{0, 1, \ldots, 19\}\}.$$

Row 0 is the *top*, row 19 is the bottom, column 0 is the left, and column 19 is the right. The obstacle set is $\mathcal{O} = O_1 \cup \cdots \cup O_5$, where

$$\begin{aligned}
O_1 &= \{(r, 5) : 0 \le r \le 15,\ r \notin \{4, 12\}\},\\
O_2 &= \{(r, 10) : 4 \le r \le 19,\ r \notin \{8, 16\}\},\\
O_3 &= \{(r, 15) : 0 \le r \le 15,\ r \notin \{6, 13\}\},\\
O_4 &= \{(9, c) : 6 \le c \le 9,\ c \neq 8\},\\
O_5 &= \{(14, c) : 11 \le c \le 14,\ c \neq 13\}.
\end{aligned}$$

There are 48 obstacle cells and 352 traversable cells. The state space is

$$\mathcal{S} = \mathcal{C} \setminus \mathcal{O}.$$

Every episode starts in the fixed state $s_0 = (19, 0)$. The gold state is $g = (0, 19)$. It is the only terminal state. Obstacles are not states and the agent can never occupy one.

The following picture is redundant with the equations above; the equations are authoritative. Rows are shown from 0 through 19. Here `S` is the start, `G` is the gold, `#` is an obstacle, and `.` is traversable.

```
.....#.........#...G
.....#.........#....
.....#.........#....
.....#.........#....
..........#....#....
.....#....#....#....
.....#....#.........
.....#....#....#....
.....#.........#....
.....###.##....#....
.....#....#....#....
.....#....#....#....
..........#....#....
.....#....#.........
.....#....###.##....
.....#....#....#....
....................
..........#.........
..........#.........
S.........#.........
```

2.2 Actions

The action space is $\mathcal{A} = \{0, 1, 2, 3\}$. The numbering and displacement vectors are fixed:

Action	Meaning	$\Delta(a)$
0	up	$(-1, 0)$
1	right	$(0, 1)$
2	down	$(1, 0)$
3	left	$(0, -1)$

2.3 Collision rule

For a nonterminal state $s = (r, c)$ and a direction b, define

$$m(s, b) = \begin{cases} s + \Delta(b), & \text{if } s + \Delta(b) \in \mathcal{S}, \\ s, & \text{otherwise.} \end{cases}$$

Thus, moving beyond the edge of the grid or into an obstacle leaves the agent in its current state. A collision still consumes one environment step and receives the ordinary step reward.

2.4 Ice and transition probabilities

If the agent requests $A_t = a$, the executed direction B_t is

$$\mathbb{P}[B_t = b \mid A_t = a] = \begin{cases} 0.80, & b = a, \\ 0.10, & b = (a-1) \bmod 4, \\ 0.10, & b = (a+1) \bmod 4, \\ 0, & \text{otherwise.} \end{cases}$$

The two 0.10 outcomes are the perpendicular directions; there is no backward-slip outcome. For $s \neq g$, the entries of the action-conditioned transition matrix are

$$P^a_{ss'} = 0.80\mathbf{1}\{s' = m(s,a)\} + 0.10\mathbf{1}\{s' = m(s,(a-1) \bmod 4)\} + 0.10\mathbf{1}\{s' = m(s,(a+1) \bmod 4)\}.$$

Here $\mathbf{1}\{E\}$ is the *indicator function* of the statement E:

$$\mathbf{1}\{E\} = \begin{cases} 1, & \text{if } E \text{ is true,} \\ 0, & \text{if } E \text{ is false.} \end{cases}$$

For fixed s, a, and s', each indicator selects whether its corresponding executed direction produces s'. For example, the first term contributes 0.80 only when $s' = m(s,a)$; otherwise, it contributes zero. If multiple executed directions produce the same s', multiple indicators equal one, so the equation automatically adds their probabilities.

For example, from $(19,0)$ with action RIGHT ($a = 1$), RIGHT is executed with probability 0.80, UP with probability 0.10, and DOWN with probability 0.10. Therefore, the next state is $(19,1)$ with probability 0.80, $(18,0)$ with probability 0.10, and $(19,0)$ with probability 0.10 because the DOWN slip hits the boundary.

A second example shows why probabilities sometimes combine. From $(19,0)$ with action DOWN ($a = 2$), the intended DOWN move is blocked with probability 0.80. The LEFT slip is also blocked with probability 0.10, while the RIGHT slip moves to $(19,1)$ with probability 0.10. Thus,

$$P^2_{(19,0),(19,0)} = 0.80 + 0.10 = 0.90, \qquad P^2_{(19,0),(19,1)} = 0.10.$$

For the mathematical MDP, the goal is absorbing:

$$P^a_{gg} = 1 \qquad \text{for every } a \in \mathcal{A}.$$

Gymnasium callers must nevertheless call `reset()` after an episode ends.

2.5 Reward and discount

The realized immediate reward follows the lecture's R_{t+1} convention:

$$R_{t+1} = \begin{cases} 0, & S_t = g, \\ +100, & S_t \neq g \text{ and } S_{t+1} = g, \\ -1, & \text{otherwise.} \end{cases}$$

Consequently, the expected immediate reward function $\mathcal{R}$ is

$$R^a_s = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a] = \begin{cases} 0, & s = g, \\ 100P^a_{sg} + (-1)(1 - P^a_{sg}), & s \neq g. \end{cases}$$

For example, suppose $S_t = (0, 18)$ and the agent chooses RIGHT ($A_t = 1$). The intended RIGHT direction is executed with probability 0.80, producing $S_{t+1} = g = (0, 19)$. On that transition, and only on that transition, the agent receives $R_{t+1} = +100$ and the episode terminates. With probability 0.10, the UP slip hits the boundary and leaves the agent at $(0, 18)$; with probability 0.10, the DOWN slip moves the agent to $(1, 18)$. Each slip receives $R_{t+1} = -1$. Thus, for this state-action pair,

$$R^1_{(0,18)} = 0.80(100) + 0.20(-1) = 79.8.$$

The reward depends on the state actually reached, not merely on choosing an action that points toward the gold.

There is no separate collision penalty. The $+100$ reward replaces the -1 step reward on the transition that enters the gold cell. The discount factor is $\gamma = 0.99$. Gymnasium's `step` method returns the immediate reward but does not return the discount factor; the learning algorithm stores and uses γ.

3 Required Gymnasium contract

Use Python 3.10 or later and Gymnasium 1.x. Your class must inherit from `gymnasium.Env`. The official Gymnasium API separates termination caused by the MDP from truncation caused by an outside limit.[1]

3.1 Spaces and observation encoding

Set

```
self.action_space = gymnasium.spaces.Discrete(4)
self.observation_space = gymnasium.spaces.Discrete(400)
```

Encode (r, c) as the Python integer

$$\text{encode}(r, c) = 20r + c.$$

Decode with $r = \lfloor \text{obs}/20 \rfloor$ and $c = \text{obs} \bmod 20$. Hence the start observation is 380 and the goal observation is 19. The 48 integer IDs corresponding to obstacles are valid members of the declared `Discrete(400)` space but are never emitted by a valid reset or transition. This deliberate superset keeps the row-major encoding simple.

Expose the fixed map as read-only-by-convention attributes named `obstacles`, `start`, and `goal`. Their values should be a `frozenset` of coordinate tuples and two coordinate tuples, respectively.

3.2 Constructor and reset

The constructor signature must be

```
def __init__(self, render_mode=None):
```

Allowed render modes are `None`, `"ansi"`, and `"human"`. Implement

```
def reset(self, *, seed=None, options=None):
```

The first operational line of `reset` must call `super().reset(seed=seed)` so that `self.np_random` is seeded correctly.[2] The supplied `options` may be ignored; it must not change the fixed start. Every reset returns

[1] Gymnasium environment API: https://gymnasium.farama.org/api/env/
[2] Gymnasium custom-environment guide: https://gymnasium.farama.org/main/tutorials/environment_creation/

```
(380, {"position": (19, 0), "steps": 0})
```

except that a Python-equivalent integer type is acceptable for the observation.

3.3 Step

Implement the current five-value Step API:

```
observation, reward, terminated, truncated, info = env.step(action)
```

For each valid action, the base environment must:

1. use `self.np_random`, and not a global NumPy or Python RNG, to sample the executed direction from probabilities $(0.80, 0.10, 0.10)$;
2. apply the collision rule and update the state;
3. increment an internal step counter;
4. return the encoded next state and the reward as a Python `float`;
5. set `terminated=True` exactly when the next state is the goal;
6. return `truncated=False`; truncation belongs to the wrapper described below;
7. return an `info` dictionary containing at least the following required entries (additional diagnostic keys are allowed):

Key	Meaning
`position`	next position as the tuple `(row, column)`
`steps`	number of transitions since the most recent reset
`executed_action`	sampled direction B in $\{0, 1, 2, 3\}$
`slipped`	Boolean equal to `executed_action != action`
`blocked`	Boolean: the attempted move in the executed direction hit an edge or rock

Reject an action for which `action_space.contains(action)` is false by raising a clear exception. Calling `step` after `terminated or truncated` is outside the required interface; client code must reset first.

3.4 Exact transition-query helper

In addition to the Gymnasium methods, implement

```
def transition_probabilities(self, state, action):
    """Return {next_observation: probability}, with duplicate outcomes merged."""
```

The input `state` and dictionary keys use the integer observation encoding. Every returned probability must be positive and the probabilities must sum to one within numerical tolerance. On the goal state it returns `{19: 1.0}`. It must reject obstacle-state IDs, out-of-range state IDs, and invalid actions. Keep this method free of random sampling. The `step` logic and this method must represent the same transition kernel.

3.5 Termination versus the 500-step time limit

Register the class with

```
register(
    id="IcyGridWorld-v0",
    entry_point="icy_gridworld.env:IcyGridWorldEnv",
    max_episode_steps=500,
)
```

The `max_episode_steps` argument causes `gymnasium.make` to add a `TimeLimit` wrapper.[3] The wrapper returns `truncated=True` on transition 500 even if the gold has not been reached. Reaching the gold sets `terminated=True`. If gold is reached on transition 500, both flags may be true: they report two distinct facts. The time limit is an evaluation safeguard outside the MDP, so it does not change $\mathcal{P}$, $\mathcal{R}$, or the definition of a terminal state.

3.6 Seeding and reproducibility

Two newly created environments that are reset with the same seed and receive the same action sequence must produce identical observations, rewards, flags, and required `info` values. Different seeds are allowed to produce the same short trajectory by chance. Calling `reset(seed=None)` continues the existing random-number stream; it does not promise a newly randomized stream.

The environment seed controls ice slips only. If a demonstration samples random actions, seed the action space separately when reproducible actions are desired.

3.7 Rendering

Set

```
metadata = {"render_modes": ["ansi", "human"], "render_fps": 4}
```

For `render_mode="ansi"`, `render()` returns a string with 20 rows of 20 characters, separated by newline characters. Use `A` for the agent, `G` for uncollected gold, `#` for obstacles, and `.` for empty traversable cells. If the agent is on the gold, show `*`. A final newline is optional. For `"human"`, print the same representation. No graphical window or `rgb_array` mode is required.

3.8 GitHub submission requirement

Upload the complete implementation to a GitHub repository before the deadline and submit the repository URL through the course submission system. The repository may be public or private; if it is private, grant the instructor access before submitting. It must contain the `icy_gridworld` package, automated tests, `demo.py`, `requirements.txt`, `AI_LOG.md`, and a `README.md` with installation, test, and demonstration commands. Use meaningful commits during development rather than uploading the entire project in one final commit. Do not commit credentials, API keys, virtual environments, or generated cache files.

[3] Gymnasium registration API: https://gymnasium.farama.org/api/registry/